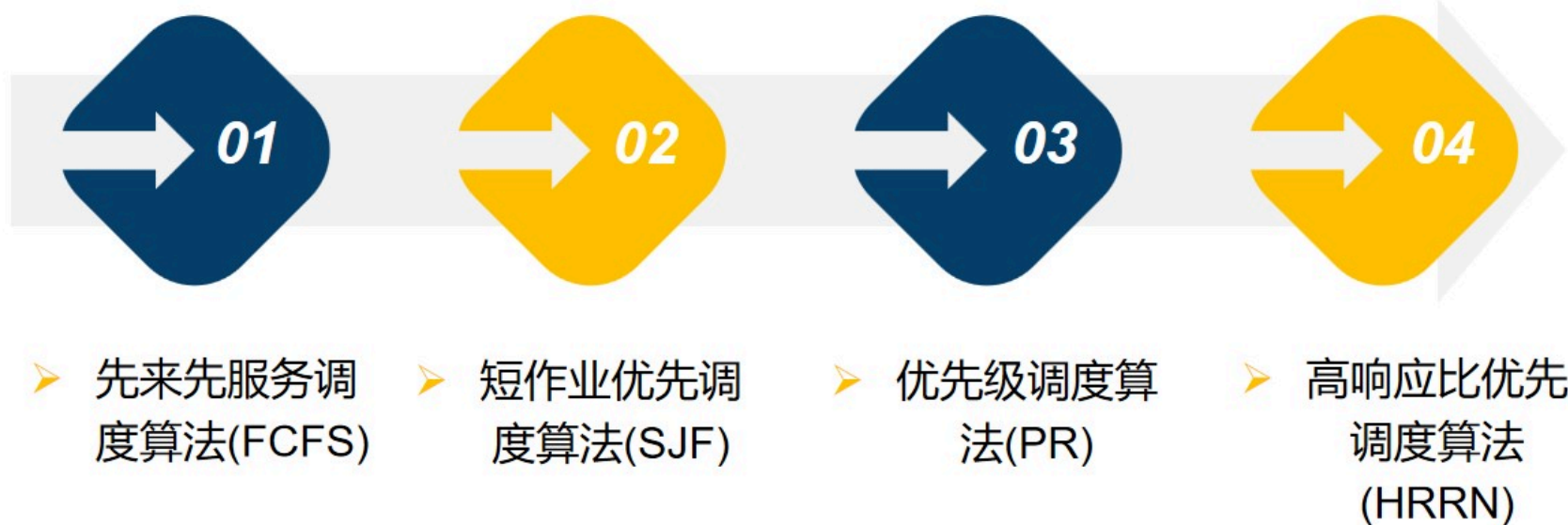


Part 2 调度算法

作业调度算法



FCFS、SJF、PR既可用于作业调度，也可用于进程调度

Part 2 调度算法

进程调度算法

 先来先服务调度算法(FCFS)

 多级队列调度算法

 短作业优先调度算法(SJF)

 多级反馈队列调度算法

 优先权调度算法(PR)

 基于公平原则的调度算法

 时间片轮转调度算法(RR)

Part 2 调度算法

2.1 先来先服务调度算法

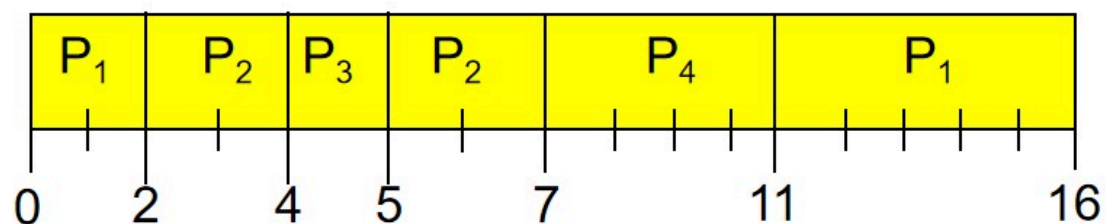
■ Q1: 先来先服务算法有何优缺点?

Part 2 调度算法

2.2 短作业优先调度算法

抢占式

进程	到达时间	区间时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4



➤ 平均等待时间 = $(9 + 1 + 0 + 2)/4 = 3$ ➤ 平均周转时间 = $(16 + 5 + 1 + 6)/4 = 7$

Part 2 调度算法

2.3 优先级调度算法

-  既可用于作业调度，也可用于进程调度。
-  基于作业/进程的紧迫程度，由外部赋予作业相应的优先级，调度算法根据优先级进行调度。
 - 每个进程都有一个优先数，优先数为整数。
 - 默认：小的优先数具有高优先级。

Part 2 调度算法

2.4 高响应比优先调度算法

➤ 响应比 $R_p = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$

Part 2 调度算法

2.4 高响应比优先调度算法

■ Q3: 高相应比优先算法有何优缺点?

- 如等待时间相同, 取决于运行时间, 类似于SJF
- 如运行时间相同, 取决于等待时间, 类似于FCFS
- 长作业可随其等待时间的增加而提高, 也可得到服务
- 缺点: 每次调度之前, 都需要计算响应比, 增加系统开销

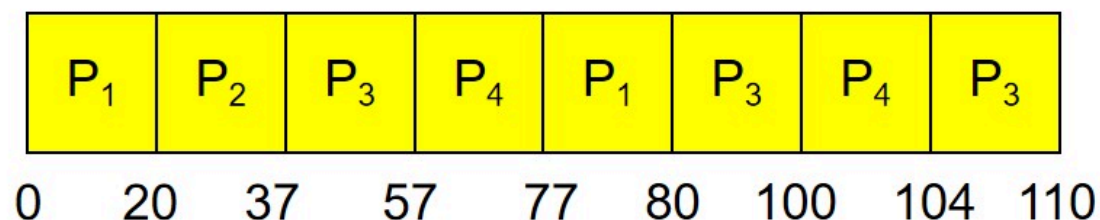
Part 2 调度算法

2.5 时间片轮转调度算法

时间片为20的RR例子

进程	运行时间
P1	23
P2	17
P3	46
P4	24

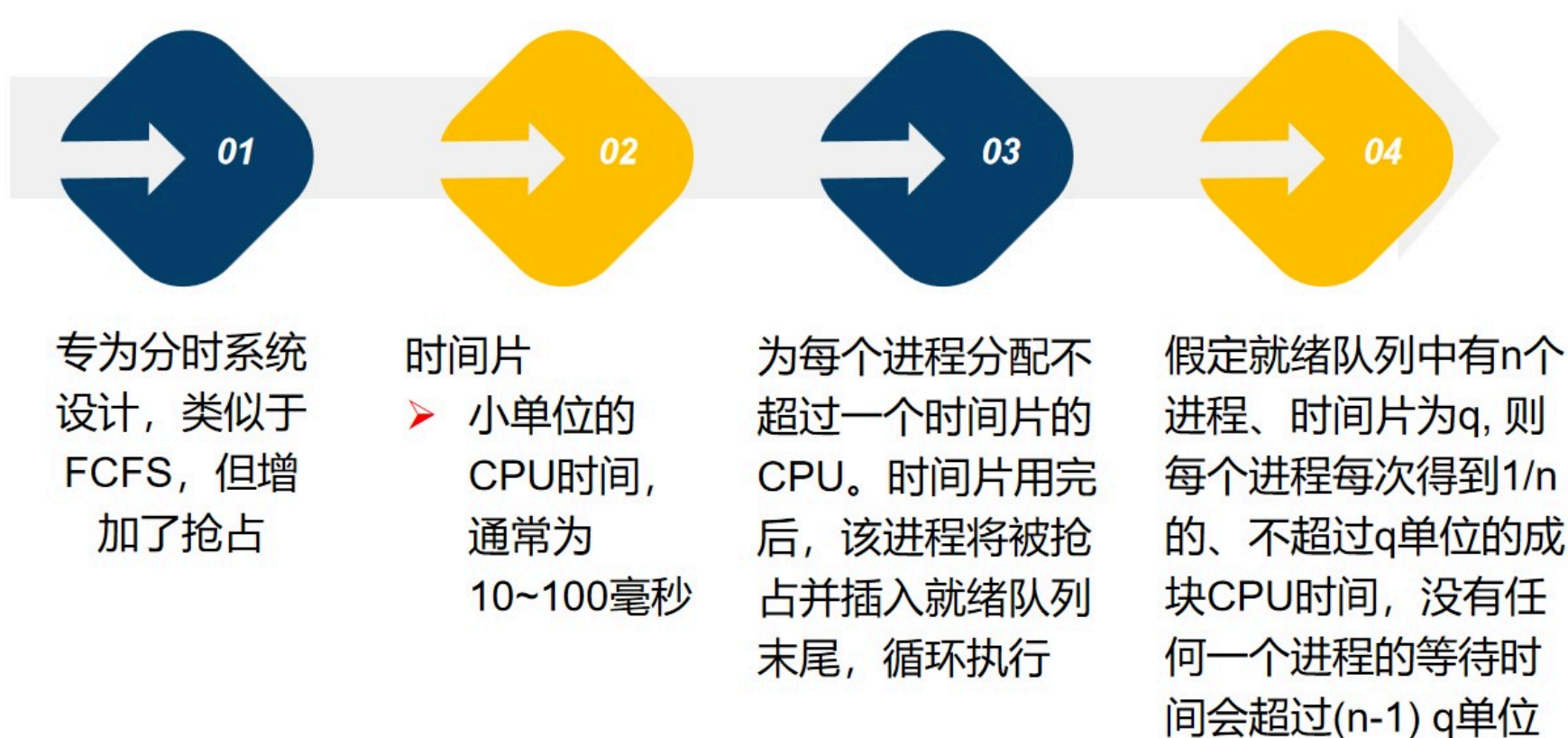
➤ Gantt图如下:



- 平均等待时间: $(57+20+64+80)/4 = 55.25$
- 平均响应时间: $(0+20+37+57)/4 = 28.5$
- 通常, RR的平均周转时间比SJF长, 但响应时间要短一些.

Part 2 调度算法

2.5 时间片轮转调度算法



Part 2 调度算法

2.5 时间片轮转调度算法

时间片大小的确定

特性

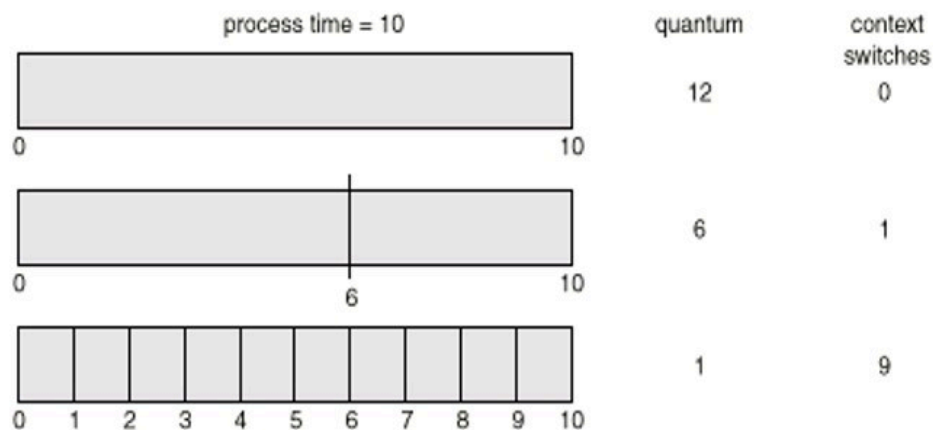
- q 大 FCFS
- q 小 增加上下文切换的时间

时间片设置应考虑

- 系统对响应时间的要求
- 就绪队列中进程的数目
- 系统的处理能力



一般准则： 时间片/10 > 进程上下文切换时间



Part 2 调度算法回顾

Q1: 处理机有哪三级调度? 各有何功能?

Q2: 调度算法指标有哪些?

Q3: 调度算法有哪些? 有何特点?

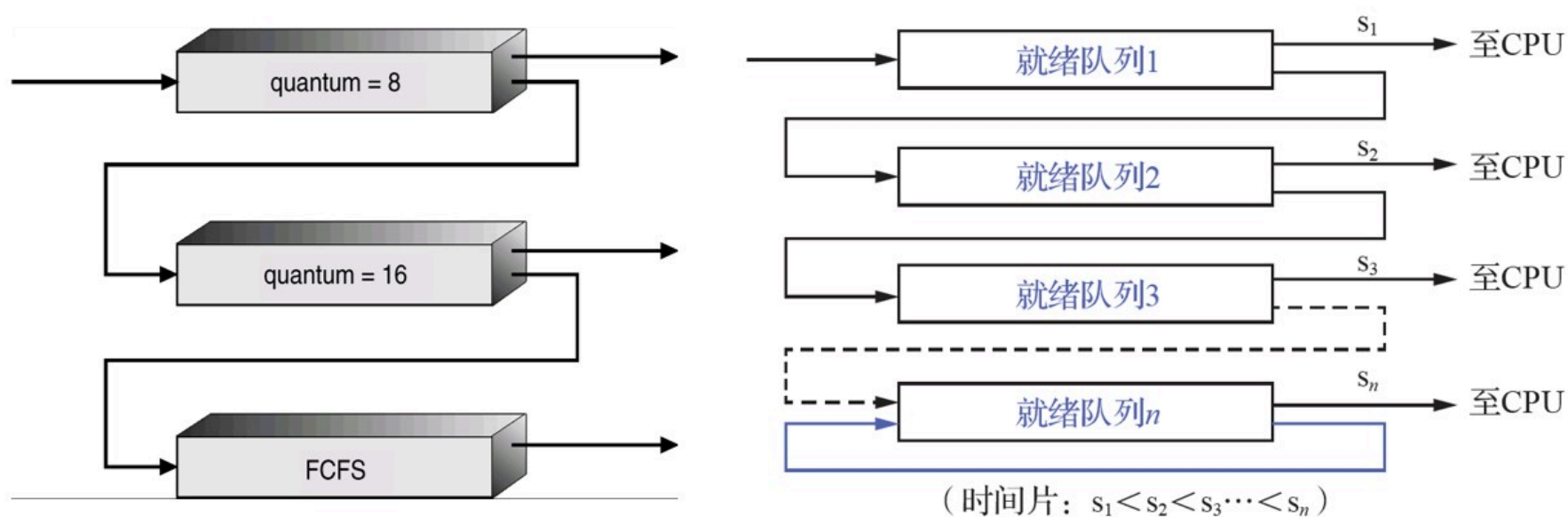
Part 2 调度算法

2.6 多级队列调度算法

- 🏠 就绪队列从一个分为多个，如：
 - 前台[交互式] ➤ 后台[批处理]
- 每个队列有自己的调度算法
 - 前台 – RR ➤ 后台 – FCFS
- 🔒 调度须在队列间进行
 - 固定优先级调度，即前台运行完后再运行后台，有可能产生饥饿。
 - 给定时间片调度，即每个队列得到一定的CPU时间，进程在给定时间内执行；如80%的时间执行前台的RR调度，20%的时间执行后台的FCFS调度

Part 2 调度算法

2.7 多级反馈队列调度算法



Part 2 调度算法

2.7 多级反馈队列调度算法

进程进队列方式

- ❖ 新进程插入第一级就绪队列末尾;
- ❖ 若进程执行过程中时间片到还没有完成, 则插入下一级就绪队列;
- ❖ 若进程执行过程中时间片未到别抢占了CPU, 则插入原就绪队列。

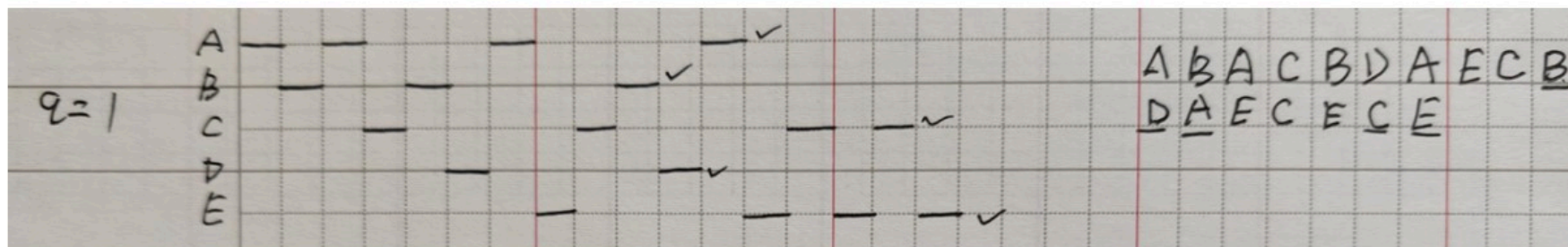
Part 2 调度算法

调度算法应用实例

有五个进程，到达就绪队列的时间分别为0, 1, 2, 3, 4, 要求的CPU服务时间分别为4, 3, 4, 2, 4, 分析按照RR ($q=1$) 和MFQ ($q=2^{i-1}$) 进程调度算法调度时，各进程的完成时间和周转时间以及带权周转时间。

Part 2 调度算法

调度算法应用实例



Part 2 调度算法

调度算法总结

进程调度的时机有哪些？

每种算法的特点是？

处理机的调度有（ ）。

☒ A 进程调度

☒ B 作业调度

☒ C 对换调度

☐ D 实时调度

以下会引起进程调度的有（ ）。

- ☒ A 当前进程执行结束 ✓
- ☒ B 当前进程时间片到 ✓
- ☒ C 当前进程阻塞 ✓
- ☒ D 就绪队列中有优先级更高的进程到达 ✓

关于调度算法描述正确的是（ ）。

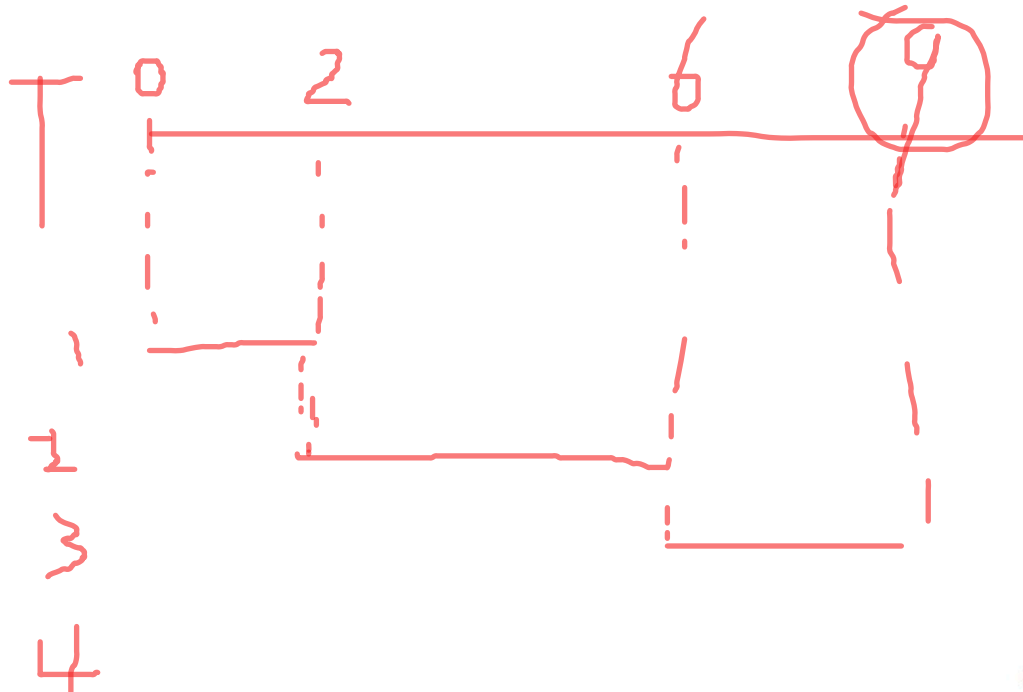
- ☐ A 短作业优先算法平均周转时间最短
- ☐ B 先来先服务算法对短作业有利
- ☒ C 时间片轮转算法每个进程按照到达顺序调度每次获得一个时间片
- ☐ D 高响应比优先调度算法中进程到达越早响应比越高

下列进程调度算法中，综合考虑进程等待时间和执行时间的是（ ）。

- ☐ A 时间片轮转算法
- ☐ B 短进程优先调度算法
- ☐ C 先来先服务调度算法
- ☒ D 高响应比优先调度算法

有四个作业，到达时间分别为0, 1, 5, 6, 服务时间分别为2, 4, 3, 1, 按照先来先服务算法，作业3的周转时间为（ ）。

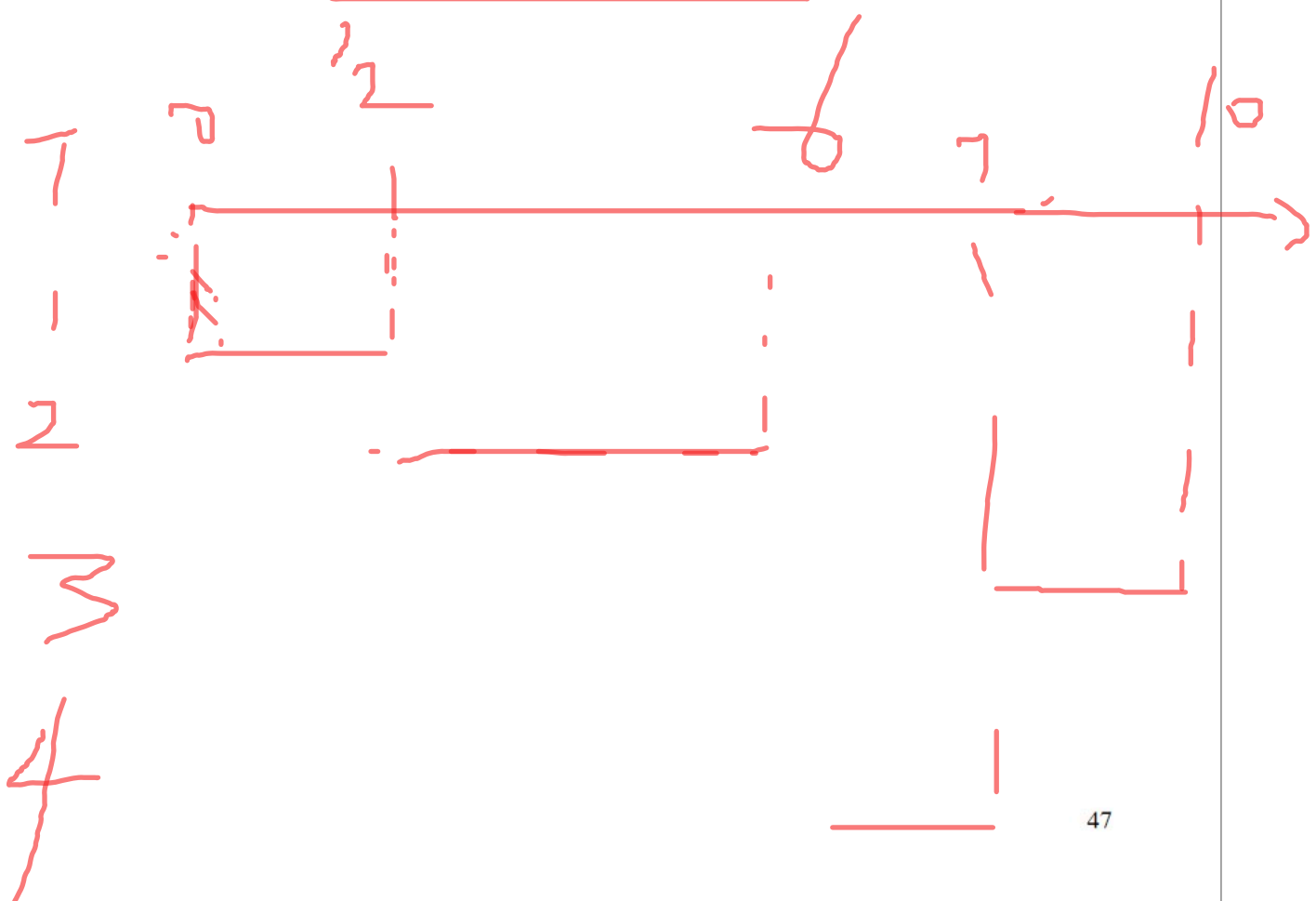
- ☒ A 4
- ☐ B 5
- ☐ C 6
- ☐ D 10



单选题 1分

有四个作业，到达时间分别为0, 1, 5, 6, 服务时间分别为2, 4, 3, 1, 按照非抢占短作业优先算法，作业3的带权周转时间为（ ）。

- A 1
- B 1.67**
- C 3
- D 3.33



单选题 1分

一作业 8 : 00 到达系统, 估计运行时间为 1 小时, 若 10 : 00 开始执行该作业, 其响应比是 () 。

A 2

B 1

C 3

D 0.5

$$1 + \frac{2}{1}$$

有三个同时到达的作业J1、J2、J3，运行时间分别是1、2、3，若系统中仅有一台单道运行的处理机，按照（ ）序列执行时平均周转时间最小。

- ☒ A J1-->J2-->J3
- ☐ B J2-->J3-->J1
- ☐ C J2-->J1-->J3
- ☐ D J1-->J3-->J2

三个同时到达的作业J1、J2 和J3，执行时间分别是T1、T2 和T3
($T1 < T2 < T3$)，若系统按照短作业优先的调度算法，则平均
周转时间为 ()。

- A $T1+T2+T3$
- ☒ B $(3T1+2T2+T3) / 3$
- C $(T1+T2+T3) / 3$
- D $(T1+2T2+3T3) / 3$

$T1$

$T1 + T2$

$T1 + T2 + T3$

B

Part 3 死锁概念

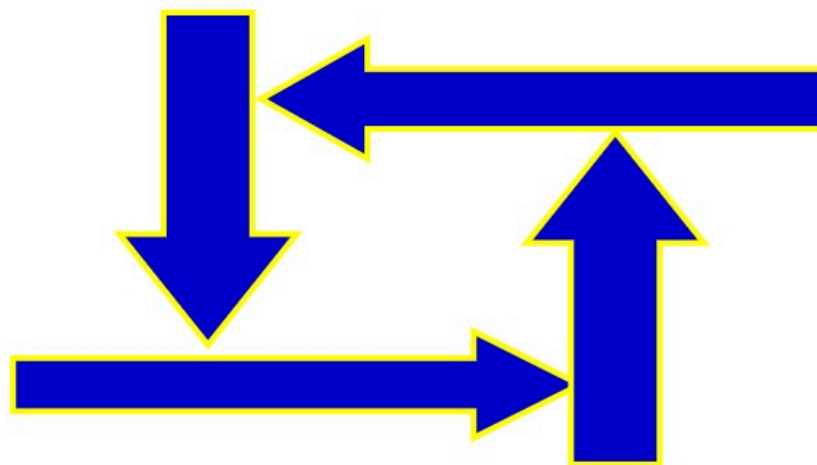


学习目标

- 能够理解死锁的定义
- 能够分析判断是否发生死锁并分析原因
- 能够理解死锁发生的必要条件

Part 3 死锁概念

3.1 死锁概念



Part 3 死锁概念

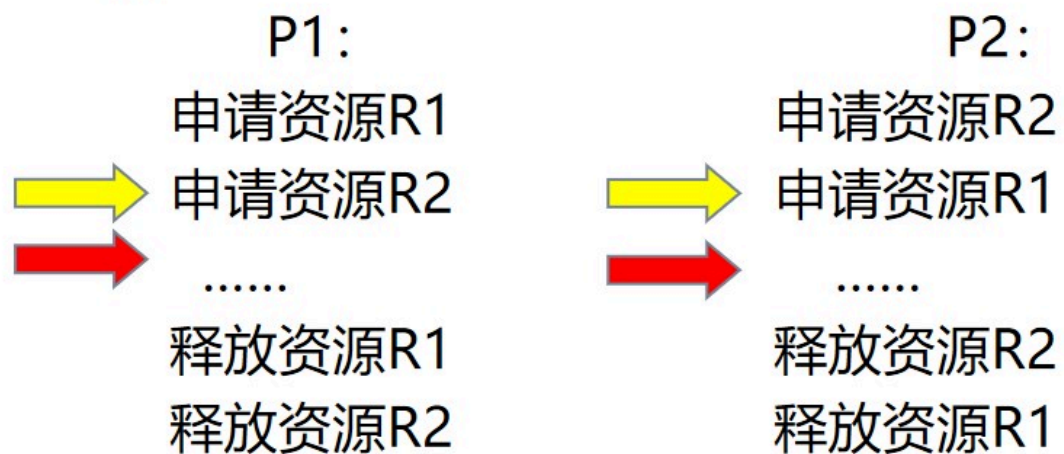
3.1 死锁概念

死锁 (Deadlock)：指多个进程在运行过程中因争夺资源而造成的一种僵局，当进程处于这种僵持状态时，若无外力作用，这些进程都将永远不能再向前推进。

Part 3 死锁概念

3.2 死锁产生的原因

- 例3-1



Part 3 死锁概念

3.2 死锁产生的原因

资源问题

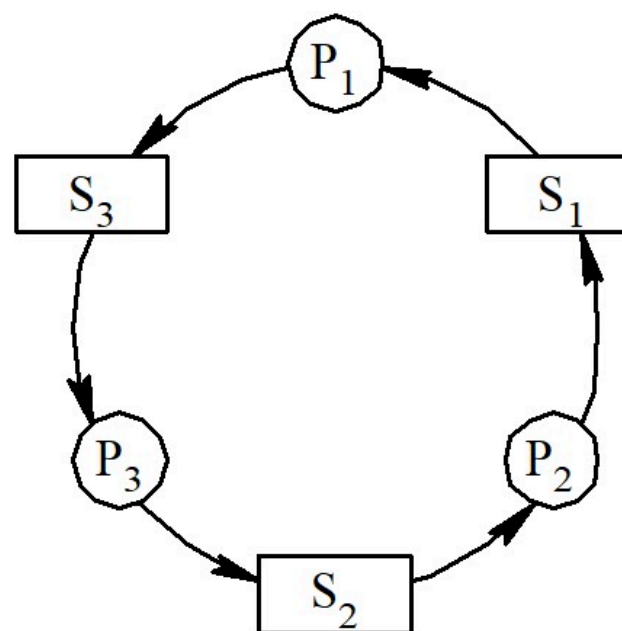
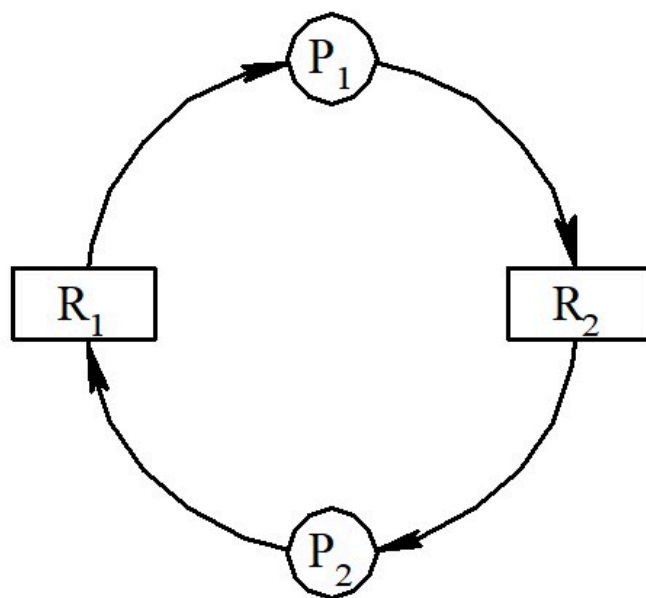


可重用性资源和可消耗性资源

- 可重用性资源：一次只能分配给一个进程，不允许多个进程共享。
- 可消耗性资源：由进程动态创建和消耗（进程间通信的消息）。

Part 3 死锁概念

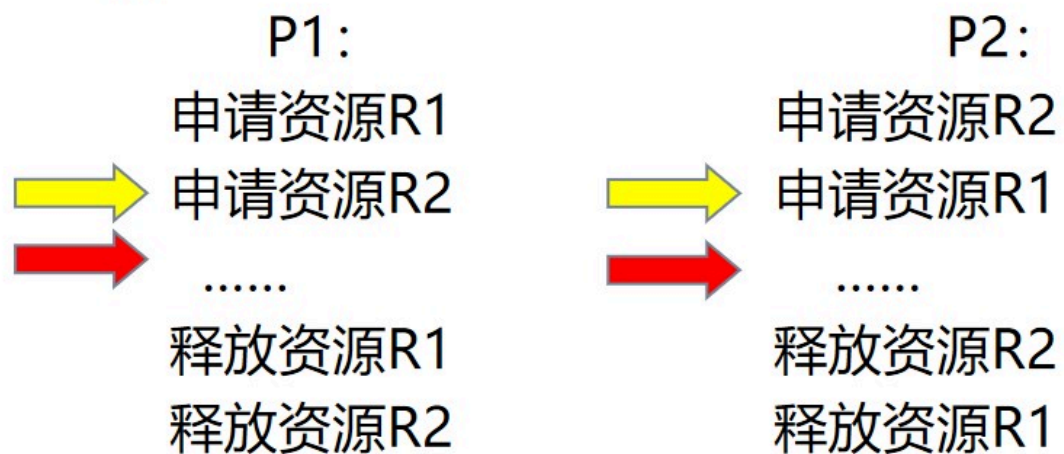
3.2 死锁产生的原因



Part 3 死锁概念

3.2 死锁产生的原因

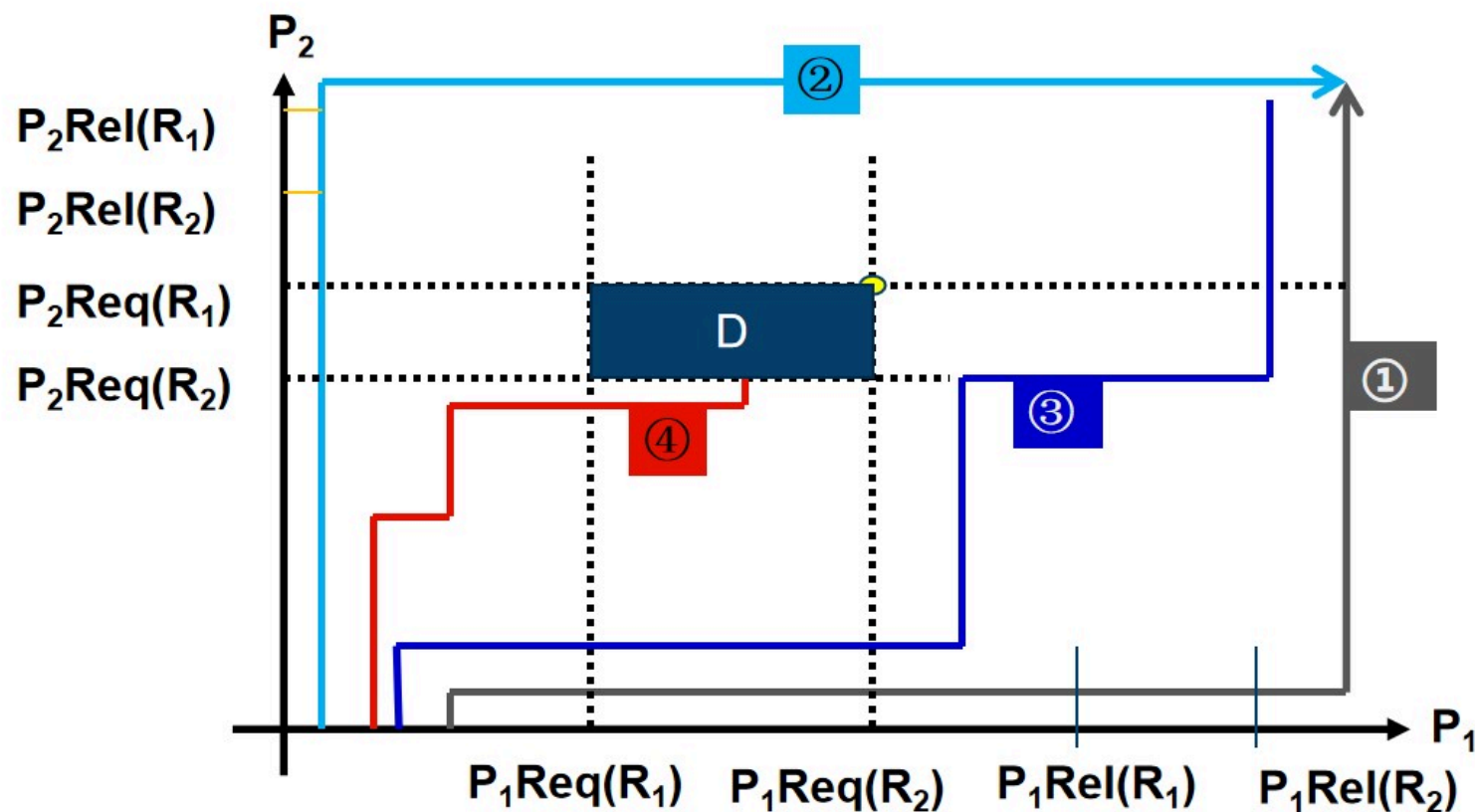
- 例3-1



Part 3 死锁概念

3.2 死锁产生的原因

进程推进顺序问题



Part 3 死锁概念

3.3 死锁的必要条件

- 互斥条件
- 请求和保持条件
- 不可抢占条件 (不可剥夺)
- 循环等待条件 (环路等待)

Part 3 死锁概念

3.3 死锁的必要条件

- **判断是否死锁的方法 (补充)**

系统中有N个进程，某种资源M个，每个进程需要最多k个资源，
系统不会发生死锁的条件是：

$$N(k-1) < M$$

Part 4 死锁处理方法



学习目标

- 能够理解死锁处理的几类方法
- 能够解释死锁预防的方法及局限性
- 能够判区分系统安全状态和不安全状态
- 能够应用银行家算法分析系统进行资源分配
- 能够应用资源分配图检测死锁

Part 4 死锁处理方法

处理死锁的方法

- 🏠 确保系统永远不会进入死锁状态
 - 死锁预防
 - 死锁避免
- 📚 允许系统进入死锁状态，然后恢复系统
 - 死锁检测 ➤ 死锁恢复
- 🔒 忽略这个问题，假装系统中从未出现过死锁。这个方法被大部分的操作系统采用，包括UNIX

Part 4 死锁处理方法

4.1 死锁的预防

- 破坏四个死锁的四个必要条件之一：

破坏“互斥”条件

破坏“请求和保持”条件

破坏“不可抢占”条件

破坏“循环等待”条件